BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC

XÂY DỰNG HỆ THỐNG NHÚNG HỖ TRỢ BÁN HÀNG THÔNG MINH

Môn học Xây dựng các hệ thống nhúng (INT1461)
Giảng viên hướng dẫn: Ths. Nguyễn Trọng Kiên
Thực hiện bởi nhóm 10:

Hồ Công Toản	**N22DCCN086**
Nguyễn Tấn Khoa	**N22DCCN042**
Đậu Văng Đăng Khoa	**N22DCCN040**
Hoàng Phi Long	**N22DCCN048**

HÀ NỘI, THÁNG 3 NĂM 2026

Lời cam đoan

Chúng tôi cam đoan rằng đồ án này là công trình nghiên cứu của nhóm chúng tôi dưới sự hướng dẫn của giảng viên. Các nội dung nghiên cứu và kết quả trong đồ án này là trung thực và chưa từng được công bố dưới bất kỳ hình thức nào. Những số liệu phục vụ cho việc phân tích, đánh giá được thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Lời cảm ơn

Chúng tôi xin gửi lời cảm ơn chân thành đến Ths. Nguyễn Trọng Kiên đã tận tình hướng dẫn và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án.

Chúng tôi cũng xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện thuận lợi để nhóm hoàn thành đồ án này.

Tóm tắt

Dự án ShopNexus xây dựng hệ thống nhúng hỗ trợ bán hàng thông minh, bao gồm thiết bị ESP32 thu âm thanh và hình ảnh, máy chủ xử lý AI (khử nhiễu DTLN, nhận dạng giọng nói qua PhoWhisper, phân loại sản phẩm bằng XLM-RoBERTa), hệ thống backend Go với kiến trúc modular monolith, tìm kiếm hybrid vector qua Milvus, và giao diện web Next.js 16. Báo cáo này trình bày tiến độ thực hiện tính đến buổi 7.

MỤC LỤC

1 Tổng quan **1**

1.1 Giới thiệu đề tài 1

1.2 Tình hình nghiên cứu 1

1.3 Mục tiêu đề tài 2

1.4 Phạm vi đề tài 2

1.5 Cấu trúc báo cáo 3

2 Cơ sở lý thuyết **4**

2.1 Vi điều khiển ESP32 4

2.1.1 Tổng quan ESP32 4

2.1.2 Kiến trúc dual-core và FreeRTOS 4

2.1.3 Camera OV5640 5

2.1.4 Microphone INMP441 5

2.1.5 Giao thức I2S 5

2.1.6 Giao thức TCP 6

2.1.7 Giao thức WebSocket 7

2.2 Mô hình khử nhiễu DTLN 7

2.2.1 Kiến trúc hai giai đoạn 7

2.2.2 Hàm mất mát Negative SNR 9

2.3 Mô hình XLM-RoBERTa 9

2.4 Mô hình PhoWhisper 9

2.5 Mô hình GPT-4o-mini 10

3 Thiết kế hệ thống **11**

3.1 Kiến trúc tổng thể 11

3.2 Luồng xử lý chính 12

3.3 Thiết kế thiết bị nhúng ESP32 13

3.3.1 Kiến trúc Dual-Core 13

3.3.2 Cấu hình chân kết nối 13

3.4 Thiết kế dịch vụ khử nhiễu 14

3.5 Thiết kế dịch vụ nhận dạng giọng nói 15

3.6 Thiết kế dịch vụ phân loại sản phẩm 16

3.7 Thiết kế server trung gian 18

4 Cài đặt hệ thống 20

4.1 Cài đặt thiết bị nhúng ESP32 20

4.1.1 Môi trường phát triển 20

4.1.2 Kết nối WiFi và khởi tạo 20

4.1.3 Cấu hình I2S 20

4.1.4 Chuẩn hóa tín hiệu 21

4.1.5 Pipeline streaming video 21

4.1.6 Giao thức điều khiển camera 21

4.2 Cài đặt dịch vụ khử nhiễu 22

4.2.1 Huấn luyện trên VIVOS 22

4.2.2 Cài đặt mô hình DTLN 23

4.2.3 Pipeline xử lý khử nhiễu 23

4.2.4 Xử lý lỗi 24

4.3 Cài đặt dịch vụ nhận dạng giọng nói 24

4.3.1 Speech-to-Text với PhoWhisper 24

4.3.2 Sinh mô tả sản phẩm với GPT-4o-mini 24

4.3.3 Quản lý phong cách viết 25

4.3.4 Công nghệ và dependencies 25

4.4 Cài đặt dịch vụ phân loại sản phẩm 25

4.4.1 Validation đầu vào 25

4.4.2 Pipeline suy luận 25

4.4.3 Containerization 26

4.4.4 Hiệu năng 26

5 Kết quả và đánh giá 27

5.1 Kết quả đạt được 27

5.2 Kết quả thiết bị nhúng ESP32 27

5.3 Kết quả khử nhiễu DTLN 28

5.4 Kết quả nhận dạng giọng nói và sinh mô tả 28

5.5 Kết quả phân loại sản phẩm 28

5.6 Kết quả tích hợp hệ thống 30

5.7 Ưu điểm 30

5.8 Hạn chế 31

6 Kết luận và hướng phát triển 32

6.1 Kết luận 32
6.2 Hướng phát triển 32

Tài liệu tham khảo **34**

CHƯƠNG 1

TỔNG QUAN

1.1 Giới thiệu đề tài

Thương mại điện tử đang phát triển mạnh mẽ tại Việt Nam với tốc độ tăng trưởng trung bình 25% mỗi năm, đưa Việt Nam trở thành một trong những thị trường thương mại điện tử phát triển nhanh nhất Đông Nam Á. Sự bùng nổ của các nền tảng như Shopee, Lazada, Tiki đã tạo điều kiện cho hàng triệu người bán nhỏ lẻ tham gia kinh doanh trực tuyến. Tuy nhiên, một trong những rào cản lớn nhất đối với người bán là việc soạn mô tả sản phẩm chất lượng — công việc đòi hỏi kỹ năng viết quảng cáo, hiểu biết về SEO và tốn nhiều thời gian.

Theo khảo sát, người bán trung bình mất 10–15 phút để soạn mô tả cho một sản phẩm, bao gồm chụp ảnh, viết tiêu đề, mô tả chi tiết và chọn danh mục phù hợp. Với những người bán có hàng trăm sản phẩm, đây là một công việc tốn kém đáng kể. Đặc biệt, nhiều người bán nhỏ lẻ tại các vùng nông thôn không thành thạo việc soạn thảo văn bản trên máy tính, nhưng lại có khả năng mô tả sản phẩm bằng lời nói một cách tự nhiên.

Dự án ShopNexus ra đời nhằm giải quyết vấn đề trên bằng cách xây dựng hệ thống nhúng hỗ trợ bán hàng thông minh, cho phép người bán chỉ cần mô tả sản phẩm bằng giọng nói thông qua thiết bị ESP32-CAM. Hệ thống tự động thực hiện toàn bộ quy trình: khử nhiễu âm thanh, nhận dạng giọng nói, sinh mô tả sản phẩm chuyên nghiệp theo nhiều phong cách viết, và phân loại vào danh mục phù hợp — tất cả chỉ với một thao tác nói.

1.2 Tình hình nghiên cứu

Bài toán hỗ trợ bán hàng bằng giọng nói liên quan đến nhiều lĩnh vực nghiên cứu đang được phát triển mạnh mẽ.

Trong lĩnh vực khử nhiễu giọng nói, các phương pháp dựa trên deep learning đã cho kết quả vượt trội so với các phương pháp truyền thống như spectral subtraction hay Wiener filtering. Mô hình DTLN (Dual-signal Transformation LSTM Network) được đề xuất bởi Westhausen và Meyer (2020) là một trong những mô hình nhỏ gọn nhất ($\approx$ 1,8 triệu tham số) mà vẫn đạt hiệu năng real-time, phù hợp cho triển khai trên thiết bị nhúng.

Về nhận dạng giọng nói tiếng Việt, các mô hình như PhoWhisper (VinAI) đã đạt được độ chính xác đáng kể, tuy nhiên yêu cầu tài nguyên tính toán đáng kể (khoảng 2 GB VRAM).

Trong phân loại sản phẩm, các mô hình transformer đa ngôn ngữ như XLM-RoBERTa cho phép phân loại text ở nhiều ngôn ngữ khác nhau mà không cần dữ liệu huấn luyện riêng cho từng ngôn ngữ, nhờ cơ chế cross-lingual transfer learning.

Điểm khác biệt của ShopNexus so với các nghiên cứu hiện có là việc tích hợp toàn bộ pipeline — từ thu thập dữ liệu trên thiết bị nhúng đến xử lý AI và hiển thị kết quả — thành một hệ thống end-to-end hoàn chỉnh, tập trung vào ngữ cảnh thương mại điện tử Việt Nam.

1.3 Mục tiêu đề tài

Đề tài hướng đến các mục tiêu cụ thể sau:

- Xây dựng thiết bị nhúng ESP32-CAM có khả năng thu âm thanh và hình ảnh sản phẩm đồng thời, truyền dữ liệu real-time đến server xử lý thông qua giao thức TCP, đảm bảo không mất mát dữ liệu trong quá trình truyền.
- Phát triển dịch vụ khử nhiễu giọng nói sử dụng mô hình DTLN, huấn luyện trên bộ dữ liệu tiếng Việt VIVOS kết hợp với dữ liệu nhiễu Microsoft DNS, đạt latency dưới 50 ms để đảm bảo xử lý real-time.
- Xây dựng dịch vụ nhận dạng giọng nói (Speech-to-Text) và tự động sinh mô tả sản phẩm theo nhiều phong cách viết khác nhau, xử lý tốt các đặc thù của tiếng Việt đa vùng miền.
- Phát triển dịch vụ phân loại sản phẩm vào 32 danh mục thương mại điện tử sử dụng mô hình XLM-RoBERTa, đạt accuracy trên 85%.
- Tích hợp toàn bộ pipeline thành hệ thống end-to-end hoạt động liền mạch với giao diện web trực quan, cho phép người dùng thao tác dễ dàng.

1.4 Phạm vi đề tài

- **Phần cứng:** Thiết bị nhúng ESP32-CAM với microphone INMP441 (thu âm 16 kHz, mono) và camera OV5640 (JPEG, hỗ trợ đến 1600×1200). Kết nối WiFi 802.11 b/g/n, truyền dữ liệu qua TCP.
- **Phần mềm:** Server trung gian Node.js (relay TCP↔WebSocket), dịch vụ khử nhiễu (Flask + TensorFlow), dịch vụ nhận dạng giọng nói (FastAPI + PhoWhisper), dịch vụ phân loại (FastAPI + PyTorch), và giao diện web.
- **Ngôn ngữ:** Tập trung xử lý tiếng Việt (đa vùng miền). Mô hình phân loại hỗ trợ đa ngôn ngữ nhờ XLM-RoBERTa.

- **Danh mục:** 32 danh mục thương mại điện tử phổ biến, từ Electronics đến Fashion, Health, và Industrial.
- **Giới hạn:** Hệ thống yêu cầu kết nối WiFi ổn định.

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành 6 chương:

- **Chương 1 – Tổng quan:** Giới thiệu đề tài, tình hình nghiên cứu, mục tiêu và phạm vi.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các kiến thức nền tảng về phần cứng ESP32, giao thức truyền thông, và các mô hình AI được sử dụng (DTLN, XLM-RoBERTa, GPT-4o).
- **Chương 3 – Thiết kế hệ thống:** Kiến trúc tổng thể, luồng xử lý, và thiết kế chi tiết từng thành phần.
- **Chương 4 – Cài đặt hệ thống:** Chi tiết cài đặt phần cứng, cấu hình các dịch vụ, và triển khai pipeline xử lý.
- **Chương 5 – Kết quả và đánh giá:** Kết quả đạt được, phân tích ưu điểm và hạn chế của hệ thống.
- **Chương 6 – Kết luận và hướng phát triển:** Tổng kết và đề xuất hướng phát triển tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Vi điều khiển ESP32

2.1.1 Tổng quan ESP32

ESP32 là dòng vi điều khiển SoC (System on Chip) do Espressif Systems phát triển, được thiết kế cho các ứng dụng IoT và thiết bị nhúng. Chip tích hợp bộ xử lý Xtensa LX6 dual-core chạy ở tần số lên đến 240 MHz, cùng với WiFi 802.11 b/g/n và Bluetooth 4.2/BLE trên một chip duy nhất. Với giá thành thấp (khoảng 2–5 USD) và hệ sinh thái phong phú, ESP32 trở thành lựa chọn phổ biến cho các dự án nhúng từ hobby đến thương mại.

Dự án sử dụng board ESP32-CAM — một biến thể tích hợp sẵn khe cắm camera và hỗ trợ PSRAM 4 MB mở rộng, cho phép xử lý ảnh và buffer dữ liệu lớn mà không bị giới hạn bởi 520 KB SRAM nội bộ.

Bảng 2.1: Thông số phần cứng ESP32-CAM

Thành phần	Thông số
Board	ESP32-CAM với PSRAM
Vi xử lý	Xtensa LX6 dual-core, 240 MHz
Camera	OV5640, JPEG, QVGA (320×240) mặc định, hỗ trợ đến UXGA (1600×1200)
Microphone	INMP441 (I2S digital), 16 kHz, mono
Kết nối	WiFi 802.11 b/g/n, 2.4 GHz
Bộ nhớ	520 KB SRAM + 4 MB PSRAM
Flash	4 MB SPI Flash
GPIO	16 chân GPIO đa chức năng
Nguồn	5V qua USB hoặc pin, 3.3V logic

2.1.2 Kiến trúc dual-core và FreeRTOS

ESP32 sử dụng hai nhân xử lý Xtensa LX6 (gọi là Core 0 và Core 1), cho phép chạy song song hai tác vụ độc lập. Hệ điều hành thời gian thực FreeRTOS được tích hợp sẵn trong ESP-

Thêm hình: Ảnh thực tế board ESP32-CAM — mặt trước hiển thị chip ESP32, khe cắm camera, ăng-ten WiFi. Mặt sau hiển thị khe cắm thẻ microSD và PSRAM chip.

Hình 2.1: Board ESP32-CAM

IDF (Espressif IoT Development Framework), cung cấp cơ chế quản lý task, semaphore, queue và timer. Mỗi task được gán vào một core cụ thể thông qua hàm $\mathrm{xTaskCreatePinnedToCore}()$, đảm bảo tác vụ thời gian thực (như thu âm) không bị preempt bởi các tác vụ khác.

2.1.3 Camera OV5640

OV5640 là cảm biến hình ảnh CMOS 5 megapixel của OmniVision, hỗ trợ nhiều độ phân giải từ QQVGA (160×120) đến QSXGA (2592×1944). Trong dự án, camera được cấu hình ở chế độ JPEG để giảm tải cho ESP32 — thay vì truyền raw pixel, camera nén ảnh trước khi gửi qua bus dữ liệu 8-bit song song. Camera giao tiếp với ESP32 qua hai giao thức: bus dữ liệu song song (8 chân D0–D7) cho truyền pixel, và I2C (SDA/SCL) cho cấu hình các thanh ghi điều khiển như độ sáng, contrast, white balance.

2.1.4 Microphone INMP441

INMP441 là microphone MEMS (Micro-Electro-Mechanical Systems) digital của InvenSense (TDK), tích hợp sẵn bộ chuyển đổi ADC và giao tiếp I2S. So với microphone analog truyền thống cần thêm mạch khuếch đại và ADC bên ngoài, INMP441 đơn giản hóa đáng kể thiết kế phần cứng. Microphone có độ nhạy -26 dBFS, SNR 61 dBA, và dải tần đáp ứng 60 Hz – 15 kHz, phù hợp cho thu giọng nói trong môi trường bán hàng.

2.1.5 Giao thức I2S

I2S (Inter-IC Sound) là giao thức truyền thông nối tiếp đồng bộ được thiết kế chuyên biệt cho dữ liệu âm thanh số, ban đầu được Philips (nay là NXP) đề xuất vào năm 1986. Giao thức

Thêm hình: Ảnh module microphone INMP441 — hiển thị chip MEMS, các chân kết nối (VDD, GND, SD, SCK, WS, L/R), kích thước nhỏ gọn.

Hình 2.2: Module microphone INMP441

sử dụng 3 đường tín hiệu chính:

- **SCK (Serial Clock):** Xung nhịp đồng bộ dữ liệu, tần số bằng tần số lấy mẫu × số bit × số kênh. Với cấu hình 16 kHz, 32-bit, stereo: SCK = 1.024 MHz.
- **WS (Word Select):** Chọn kênh trái (WS = 0) hoặc kênh phải (WS = 1). Tần số WS bằng tần số lấy mẫu (16 kHz).
- **SD (Serial Data):** Đường dữ liệu nối tiếp, truyền MSB first.

Microphone INMP441 hỗ trợ I2S với độ phân giải 18-bit thực tế, đóng gói trong word 32-bit với dữ liệu nằm ở các bit cao. ESP32 hoạt động ở chế độ I2S Master RX, cung cấp xung nhịp SCK và WS, nhận dữ liệu từ microphone qua SD.

Thêm hình: Giản đồ thời gian (timing diagram) giao thức I2S — hiển thị 3 tín hiệu SCK, WS, SD với dữ liệu 32-bit mỗi kênh. Chú thích MSB first, vùng dữ liệu 18-bit hữu ích và 14-bit padding.

Hình 2.3: Giản đồ thời gian giao thức I2S

2.1.6 Giao thức TCP

TCP (Transmission Control Protocol) là giao thức tầng vận chuyển đảm bảo truyền dữ liệu tin cậy, có thứ tự và không trùng lặp giữa hai thiết bị qua mạng IP. TCP sử dụng cơ chế three-way

handshake để thiết lập kết nối, sequence number để đảm bảo thứ tự, và acknowledgment kết hợp retransmission để xử lý mất gói.

Dự án chọn TCP thay vì UDP cho cả stream audio (port 3002) và video (port 3001) vì hai lý do chính: thứ nhất, mất mẫu âm thanh sẽ gây artifact trong quá trình khử nhiễu DTLN vì mô hình yêu cầu đầu vào liên tục; thứ hai, mất frame JPEG không thể khôi phục được do mỗi frame được nén độc lập. Tuy TCP có overhead cao hơn UDP, nhưng trong mạng LAN WiFi với latency thấp, sự khác biệt này không đáng kể.

2.1.7 Giao thức WebSocket

WebSocket là giao thức truyền thông full-duplex trên một kết nối TCP duy nhất, được chuẩn hóa trong RFC 6455. Khác với HTTP truyền thống (request-response), WebSocket cho phép server chủ động đẩy dữ liệu đến client mà không cần client gửi request — đặc tính quan trọng cho việc streaming real-time video và audio từ ESP32 đến trình duyệt.

Kết nối WebSocket bắt đầu bằng một HTTP Upgrade handshake, sau đó chuyển sang giao thức WebSocket với overhead header chỉ 2–14 bytes (so với hàng trăm bytes của HTTP header), giảm đáng kể bandwidth cho streaming liên tục.

2.2 Mô hình khử nhiễu DTLN

DTLN (Dual-signal Transformation LSTM Network) là mô hình khử nhiễu giọng nói được đề xuất bởi Westhausen và Meyer (2020), thiết kế cho xử lý thời gian thực trên thiết bị tài nguyên hạn chế.

2.2.1 Kiến trúc hai giai đoạn

DTLN xử lý tín hiệu qua hai giai đoạn bổ sung lẫn nhau:

Giai đoạn 1 – Miền tần số (STFT Domain): Tín hiệu đầu vào được chia thành các khung 512 mẫu, áp dụng STFT để thu được 257 bin tần số. Phổ biên độ $|X(f,t)|$ được chuẩn hóa qua InstantLayerNormalization, xử lý qua 2 lớp LSTM (128 units) với Dropout 0,25, rồi qua Dense + Sigmoid để tạo mask $M_1 \in [0,1]$. Mask được nhân với phổ biên độ gốc, kết hợp lại với phổ pha và biến đổi ngược (iFFT + Overlap-and-Add) để thu được tín hiệu $y_1(t)$.

Giai đoạn 2 – Miền đặc trưng (Feature Domain): Tín hiệu $y_1(t)$ được chiếu sang không gian 256 chiều qua Conv1D point-wise (kernel=1, stride=1). Sau InstantLayerNormalization và Separation Kernel thứ hai (cùng cấu trúc LSTM), Dense + Sigmoid tạo mask M_2 trong không gian đặc trưng. Giải mã qua Conv1D point-wise (256→512) với causal padding, rồi Overlap-and-Add tạo tín hiệu sạch $\hat{y}(t)$.

Bảng 2.2: Các thông số chính của mô hình DTLN

Thông số	Giá trị
Tần số lấy mẫu	16.000 Hz, mono
Kích thước khung (blockLen)	512 mẫu (32 ms)
Bước dịch khung (block_shift)	128 mẫu (8 ms), overlap 75%
Số bin tần số sau STFT	257 (= blockLen / 2 + 1)
Số đơn vị LSTM mỗi lớp	128 units
Số lớp LSTM mỗi giai đoạn	2 lớp
Dropout	0,25
Số bộ lọc Encoder Conv1D	256
Hàm kích hoạt mask	Sigmoid [0, 1]
Hàm mất mát	Negative SNR
Tổng số tham số	$\approx$ 1,8 triệu
Kích thước file model	3,9 MB (DTLN_vivos_best.h5)
Độ trễ xử lý	$\approx$ 32 ms (real-time capable)

Bảng 2.3: Cấu trúc Separation Kernel – Giai đoạn 1

Thành phần	Thông số	Vai trò
LSTM lớp 1	128 units, return_seq=True	Đặc trưng tần số cục bộ
Dropout	0,25	Giảm overfitting
LSTM lớp 2	128 units, return_seq=True	Phụ thuộc thời gian dài hạn
Dense	257 đơn vị	Ánh xạ về không gian bin tần số
Sigmoid	Đầu ra [0, 1]	Tạo mask $M_1(f,t)$

Bảng 2.4: So sánh hai giai đoạn xử lý DTLN

Tiêu chí	STFT (GĐ 1)	Conv1D Encoder (GĐ 2)
Loại biến đổi	Cố định (fixed basis)	Học từ dữ liệu
Chiều đầu ra	257 bin tần số	256 đặc trưng ẩn
Ý nghĩa vật lý	Rõ ràng, diễn giải được	Trừu tượng
Vai trò	Loại bỏ nhiễu tổng thể	Tinh chỉnh, phục hồi chi tiết

2.2.2 Hàm mất mát Negative SNR

Mô hình được tối ưu hóa bằng hàm mất mát Negative SNR:

$$\text{Loss} = -10 \cdot \log_{10} \left(\frac{E[s^2]}{E[(s-\hat{s})^2 + \varepsilon]} \right) \tag{2.1}$$

trong đó s là tín hiệu sạch, $\hat{s}$ là tín hiệu ước lượng, $\varepsilon = 10^{-7}$. Hàm mất mát này tối đa hóa tỷ số tín hiệu trên nhiễu (SNR) của đầu ra, trực tiếp tối ưu chất lượng âm thanh.

2.3 Mô hình XLM-RoBERTa

XLM-RoBERTa (Cross-lingual Language Model – Robustly Optimized BERT Approach) là mô hình ngôn ngữ đa ngôn ngữ được phát triển bởi Facebook AI, huấn luyện trên 2,5 TB dữ liệu CommonCrawl từ 100 ngôn ngữ.

- **Base model:** XLM-RoBERTa-base với 270M tham số.
- **Fine-tuned:** Lezh1n/xlm-roberta-ecommerce-classifier cho phân loại 32 danh mục thương mại điện tử.
- **Kích thước:** $\approx$ 1,1 GB, max sequence 512 tokens.
- **Hiệu năng:** Accuracy 90,14%, F1-score 90,00%.

Ưu điểm của XLM-RoBERTa cho bài toán này là khả năng xử lý đa ngôn ngữ (cross-lingual transfer), cho phép mô hình hiểu cả text tiếng Việt lẫn tên sản phẩm tiếng Anh thường gặp trong thương mại điện tử.

2.4 Mô hình PhoWhisper

PhoWhisper là bộ mô hình nhận dạng giọng nói tiếng Việt được phát triển bởi VinAI Research, dựa trên kiến trúc OpenAI Whisper và được fine-tune trên dữ liệu tiếng Việt quy mô lớn. Mô hình hỗ trợ nhận dạng chính xác các đặc thù ngữ âm tiếng Việt bao gồm 6 thanh điệu, các âm đặc trưng và sự khác biệt giữa giọng vùng miền (Bắc, Trung, Nam).

Dự án sử dụng phiên bản PhoWhisper-large (vinai/PhoWhisper-large), mô hình lớn nhất trong bộ PhoWhisper với kích thước khoảng 2 GB. Model được triển khai local trên server thông qua HuggingFace Transformers pipeline automatic-speech-recognition, không phụ thuộc cloud API. So với phiên bản nhỏ hơn (PhoWhisper-tiny), phiên bản large cho kết quả nhận dạng chính xác hơn đáng kể, đặc biệt với giọng vùng miền và môi trường có nhiễu.

2.5 Mô hình GPT-4o-mini

GPT-4o-mini là mô hình ngôn ngữ lớn (LLM) nhỏ gọn và tiết kiệm chi phí của OpenAI, được thiết kế cho các tác vụ không yêu cầu mô hình đầy đủ GPT-4o. Trong dự án, GPT-4o-mini được sử dụng để sinh mô tả sản phẩm từ text nhận dạng giọng nói thông qua OpenAI API.

Mô hình có khả năng hiểu ngữ cảnh sâu, cho phép xử lý hiệu quả text lộn xộn từ STT — nhận diện từ lấp ("ừm", "à"), bỏ qua các đoạn lặp lại và lỗi phát âm, trích xuất thông tin cốt lõi về sản phẩm và sinh văn bản mạch lạc theo phong cách viết được chỉ định. Với temperature 0.75 và giới hạn 600 tokens, mô hình cân bằng giữa tính sáng tạo (văn phong đa dạng) và tính nhất quán (nội dung chính xác).

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống ShopNexus được thiết kế theo kiến trúc microservices, trong đó mỗi thành phần đảm nhận một chức năng riêng biệt và giao tiếp với nhau thông qua các giao thức TCP và HTTP. Việc tách biệt các dịch vụ cho phép phát triển, triển khai và mở rộng từng thành phần một cách độc lập.

Toàn bộ hệ thống bao gồm 5 thành phần chính. Thiết bị nhúng ESP32-CAM đóng vai trò đầu vào, thu âm thanh qua microphone INMP441 và hình ảnh qua camera OV5640, sau đó truyền dữ liệu real-time đến server thông qua TCP. Server trung gian chạy trên Node.js hoạt động như cầu nối giữa thiết bị nhúng và trình duyệt, thực hiện relay TCP↔WebSocket đồng thời đóng vai trò proxy chuyển tiếp request đến các dịch vụ AI phía sau. Ba dịch vụ AI được triển khai riêng biệt: dịch vụ khử nhiễu sử dụng mô hình DTLN trên Flask, dịch vụ nhận dạng giọng nói và sinh mô tả sản phẩm trên FastAPI, và dịch vụ phân loại sản phẩm sử dụng XLM-RoBERTa cũng trên FastAPI. Figure 3.1 minh họa kiến trúc tổng thể của hệ thống.

Thêm hình: Sơ đồ kiến trúc tổng thể hệ thống — thể hiện 5 thành phần (ESP32-CAM, Node.js Server, Noise Reduction, Voice Rephraze, Product Classification) cùng các kết nối TCP/HTTP/WebSocket giữa chúng và trình duyệt.

Hình 3.1: Kiến trúc tổng thể hệ thống ShopNexus

Bảng 3.1: Tổng quan các thành phần hệ thống

Thành phần	Công nghệ	Port	Vai trò
ESP32-CAM	Arduino/C++	TCP 3001, 3002	Thu âm thanh + hình ảnh
Server trung gian	Node.js + Express	3000	Relay TCP↔WebSocket, proxy API
Noise Reduction	Flask + TensorFlow	5000	Khử nhiễu DTLN
Voice Rephraze	FastAPI + PhoWhisper	5001	STT + sinh mô tả
Product Classification	FastAPI + PyTorch	5002	Phân loại 32 danh mục

3.2 Luồng xử lý chính

Dữ liệu trong hệ thống đi qua một pipeline tuần tự gồm 5 bước. Đầu tiên, ESP32 thu âm thanh từ microphone INMP441 ở định dạng 16-bit PCM, tần số lấy mẫu 16 kHz, mono và truyền qua TCP port 3002 đến server trung gian. Tại đây, server Node.js nhận stream PCM, đóng gói thành file WAV rồi gửi đến dịch vụ khử nhiễu (port 5000) để loại bỏ tạp âm. Audio đã được làm sạch tiếp tục được chuyển đến dịch vụ Voice Rephraze (port 5001), nơi giọng nói được nhận dạng thành văn bản và tự động sinh mô tả sản phẩm theo phong cách viết do người dùng lựa chọn. Văn bản mô tả sau đó được gửi đến dịch vụ phân loại (port 5002) để xác định danh mục sản phẩm phù hợp trong 32 danh mục thương mại điện tử. Cuối cùng, toàn bộ kết quả — bao gồm spectrogram trước/sau khử nhiễu, text nhận dạng, mô tả đã sinh và top-K danh mục dự đoán — được hiển thị trên giao diện web. Figure 3.2 minh họa luồng xử lý này.

Thêm hình: Sơ đồ luồng xử lý dữ liệu (flowchart hoặc sequence diagram) — từ ESP32 thu âm → Node.js đóng gói WAV → Denoise (port 5000) → STT + sinh mô tả (port 5001) → Phân loại (port 5002) → Hiển thị kết quả trên web UI.

Hình 3.2: Luồng xử lý dữ liệu trong hệ thống

3.3 Thiết kế thiết bị nhúng ESP32

3.3.1 Kiến trúc Dual-Core

Một trong những quyết định thiết kế quan trọng nhất trên ESP32 là việc phân tách tác vụ thu âm thanh và thu hình ảnh lên hai nhân xử lý riêng biệt. Core 0 được gán riêng cho Audio Capture Task — đọc I2S liên tục từ INMP441, chuyển đổi từ 32-bit sang 16-bit PCM và stream qua TCP port 3002 với stack 16 KB. Core 1 đảm nhận Camera và Main Loop — chụp JPEG từ OV5640, gửi qua TCP port 3001, đồng thời xử lý các lệnh điều khiển camera từ server.

Bảng 3.2: Phân công xử lý dual-core trên ESP32

Core	Nhiệm vụ	Chi tiết
Core 0	Audio Capture Task	Đọc I2S liên tục từ INMP441, chuyển đổi 32-bit→16-bit PCM, stream qua TCP port 3002. Stack: 16 KB.
Core 1	Camera + Main Loop	Chụp JPEG từ OV5640, gửi qua TCP port 3001, xử lý lệnh điều khiển camera từ server.

Việc tách riêng hai core đảm bảo luồng audio chạy liên tục không bị gián đoạn bởi quá trình chụp ảnh từ camera, tránh hiện tượng buffer underrun — một vấn đề phổ biến khi xử lý đồng thời nhiều nguồn dữ liệu trên vi điều khiển.

Thêm hình: Sơ đồ kiến trúc dual-core ESP32 — Core 0 chạy Audio Task (I2S INMP441 → TCP 3002), Core 1 chạy Camera Task (OV5640 → TCP 3001) + xử lý lệnh điều khiển. Thể hiện rõ sự phân tách tác vụ giữa hai nhân.

Hình 3.3: Kiến trúc dual-core trên ESP32

3.3.2 Cấu hình chân kết nối

Camera OV5640 sử dụng bus dữ liệu 8-bit song song với các chân D0–D7 được ánh xạ tới GPIO 5, 18, 19, 21, 36, 39, 34, 35. Các tín hiệu điều khiển bao gồm XCLK (GPIO 0), PCLK

(GPIO 22), VSYNC (GPIO 25), HREF (GPIO 23), và giao tiếp I2C qua SDA (GPIO 26), SCL (GPIO 27). Chân PWDN (GPIO 32) dùng để điều khiển chế độ tiết kiệm năng lượng.

Microphone INMP441 kết nối qua giao thức I2S với 3 chân: WS – Word Select (GPIO 2), SCK – Serial Clock (GPIO 14), và SD – Serial Data (GPIO 15), sử dụng I2S port 1 để tránh xung đột với các ngoại vi khác trên port 0.

Thêm hình: Sơ đồ đấu nối phần cứng (wiring diagram) — ESP32-CAM kết nối với camera OV5640 (bus 8-bit, I2C) và microphone INMP441 (I2S: WS, SCK, SD). Ghi rõ số GPIO cho từng chân.

Hình 3.4: Sơ đồ kết nối phần cứng ESP32-CAM

3.4 Thiết kế dịch vụ khử nhiễu

Dịch vụ khử nhiễu được thiết kế theo kiến trúc layered rõ ràng, tách biệt các tầng xử lý để dễ bảo trì và mở rộng. Tầng API (`api/routes.py`) tiếp nhận HTTP request qua Flask Blueprint, thực hiện validate file đầu vào và trả response. Tầng Service (`services/noise_reduction.py`) chứa logic nghiệp vụ: đọc file WAV, tiền xử lý audio (chuyển stereo sang mono, normalize sang float32), gọi model inference, hậu xử lý kết quả (clip output, ghi file). Tầng Model (`services/dtln_model.py`) triển khai kiến trúc DTLN bằng TensorFlow/Keras, xây dựng graph tính toán và load trọng số.

Dịch vụ cung cấp hai endpoint chính. Endpoint `POST /denoise` nhận file WAV qua multipart/form-data (tối đa 50 MB), xử lý khử nhiễu và trả về file WAV đã làm sạch. Endpoint `GET /health` trả về trạng thái dịch vụ bao gồm: tình trạng model (ready/not ready), đường dẫn file trọng số, sample rate, và input/output shape của model.

Mô hình DTLN được quản lý theo Singleton Pattern thông qua phương thức `__new__()` — model chỉ được load một lần duy nhất vào bộ nhớ khi request đầu tiên đến, các request sau tái sử dụng cùng instance. Điều này tránh chi phí khởi tạo lặp lại (mỗi lần load mất 2–3 giây) và đảm bảo sử dụng bộ nhớ hiệu quả.

Framework được chọn là Flask 2.3+ kết hợp TensorFlow 2.13+. Flask phù hợp với yêu cầu xử lý đồng bộ của pipeline khử nhiễu — mỗi request cần chờ model xử lý xong trước khi trả kết quả. CORS được bật cho tất cả origins để cho phép gọi từ ESP32 và web client. Ứng dụng sử dụng Factory Pattern (create_app()) để khởi tạo, hỗ trợ cấu hình development và production riêng biệt.

Thêm hình: Sơ đồ kiến trúc layered dịch vụ khử nhiễu — 3 tầng: API Layer (Flask Blueprint: /denoise, /health) → Service Layer (NoiseReductionService: đọc WAV, tiền xử lý, hậu xử lý) → Model Layer (DTLN Singleton: STFT, LSTM, iFFT). Thể hiện luồng request từ client qua từng tầng.

Hình 3.5: Kiến trúc dịch vụ khử nhiễu

3.5 Thiết kế dịch vụ nhận dạng giọng nói

Dịch vụ Voice Rephraze (AI Copywriting Service) đảm nhận hai chức năng chính: chuyển đổi giọng nói thành văn bản (Speech-to-Text) và sinh mô tả sản phẩm theo phong cách viết tùy chọn. Toàn bộ dịch vụ được triển khai trong một file duy nhất (main.py) trên FastAPI, sử dụng AsyncOpenAI client cho các lời gọi API bất đồng bộ và HuggingFace Transformers cho xử lý giọng nói local.

Module STT sử dụng mô hình PhoWhisper-large (vinai/PhoWhisper-large) — mô hình nhận dạng giọng nói tiếng Việt do VinAI Research phát triển, chạy trực tiếp trên server thay vì gọi cloud API. PhoWhisper được huấn luyện đặc biệt cho ngữ âm và thanh điệu tiếng Việt, hỗ trợ 9 định dạng audio: WAV, MP3, FLAC, OGG, M4A, WebM, MP4, MPEG và MPGA. Model có kích thước khoảng 2 GB và được load vào VRAM khi khởi động thông qua HuggingFace pipeline automatic-speech-recognition.

Module sinh mô tả sử dụng GPT-4o-mini thông qua OpenAI API với temperature 0.75 và giới hạn 600 tokens. System prompt được thiết kế cẩn thận để AI đóng vai chuyên gia copywriter thương mại điện tử, với các ràng buộc cụ thể: phân tích văn bản đầu vào từ voice-to-text (thường lộn xộn, có từ lấp như "à", "ừm", lặp lại và lỗi phát âm), trích xuất thông tin cốt lõi về sản phẩm (tên, chất liệu, màu sắc, tính năng, lợi ích, đối tượng sử dụng), sau đó viết lại thành mô tả hoàn

chỉnh 3–5 câu ngắn gọn nhưng thuyết phục. Một quy tắc quan trọng là AI không được bịa đặt thông tin — chỉ sử dụng những gì được đề cập trong đầu vào.

Hệ thống hỗ trợ nhiều phong cách viết mô tả, được lưu trữ và quản lý trong SQLite. Khi người dùng chọn một phong cách, hệ thống truy vấn mô tả chi tiết của phong cách từ database và đưa vào prompt gửi cho GPT-4o-mini, hướng dẫn AI viết theo đúng giọng văn yêu cầu. Nếu phong cách không tồn tại trong database, hệ thống sử dụng fallback "Viết sáng tạo, tự nhiên". Table 3.3 liệt kê các phong cách mặc định được khởi tạo cùng hệ thống.

Thêm hình: Sơ đồ luồng xử lý dịch vụ Voice Rephraze — Audio đầu vào → PhoWhisper-large (STT local) → raw text → GPT-4o-mini + system prompt + style từ SQLite → mô tả sản phẩm hoàn chỉnh. Thể hiện cả luồng quản lý phong cách viết qua API.

Hình 3.6: Luồng xử lý dịch vụ nhận dạng giọng nói

Bảng 3.3: Các phong cách viết mô tả sản phẩm mặc định

Phong cách	Mô tả
Văn minh	Lịch sự, trang trọng, sử dụng từ ngữ chuẩn mực
Chuyên nghiệp	Ngôn ngữ kỹ thuật, rõ ràng, tập trung thông số sản phẩm
Thân thiện	Ấm áp, gần gũi, dễ tiếp cận người đọc
Văn hóa người Tày	Yếu tố văn hóa dân tộc Tày, hình ảnh đặc trưng
Funny	Giọng văn trẻ trung, vui tươi, cách diễn đạt mới lạ

3.6 Thiết kế dịch vụ phân loại sản phẩm

Dịch vụ phân loại sản phẩm (E-Commerce Product Classifier API, version 2.0.0) được xây dựng trên FastAPI 0.115 với PyTorch 2.5.1 và HuggingFace Transformers 4.46.3. Kiến trúc gồm hai module chính: `main.py` chứa toàn bộ FastAPI application (endpoints, Pydantic models, model lifecycle), và `categories.py` định nghĩa hệ thống phân loại 32 danh mục với ánh xạ hai chiều name↔ID.

Bảng 3.4: API endpoints dịch vụ Voice Rephraze

Endpoint	Chức năng
POST /stt	Upload audio, trả về text nhận dạng (PhoWhisper)
POST /gen	Sinh mô tả sản phẩm theo phong cách (GPT-4o-mini)
GET /styles	Danh sách tất cả phong cách viết
POST /add-style	Thêm phong cách viết mới

Mô hình XLM-RoBERTa (Lezh1n/xlm-roberta-ecommerce-classifier) được load khi khởi động thông qua FastAPI Lifespan Context Manager — một asynccontextmanager quản lý toàn bộ vòng đời model: khởi tạo tokenizer và model từ HuggingFace Hub, chuyển model sang thiết bị phù hợp (CUDA nếu có GPU, CPU nếu không), đặt ở chế độ model.eval() để tắt dropout và cố định batch normalization, sau đó tạo HuggingFace pipeline text-classification với top_k=None để trả về confidence score cho tất cả 32 danh mục. Khi server shutdown, GPU memory được giải phóng qua torch.cuda.empty_cache().

Dịch vụ cung cấp 8 REST endpoints được tổ chức theo 3 nhóm: General (GET /, GET /health), Classification (POST /classify, POST /classify/batch), và Categories (GET /categories, GET /categories/{id}, GET /categories/name/{name}, GET /model-info). CORS middleware được bật cho tất cả origins để cho phép gọi từ web client.

Bảng 3.5: API endpoints dịch vụ phân loại

Endpoint	Chức năng
POST /classify	Phân loại 1 sản phẩm, trả top-K predictions
POST /classify/batch	Batch tối đa 100 items (batch_size=32)
GET /categories	Danh sách đầy đủ 32 danh mục
GET /categories/{id}	Tra cứu danh mục theo ID (0–31)
GET /categories/name/{name}	Tra cứu danh mục theo tên
GET /model-info	Metadata: base model, số tham số, accuracy
GET /health	Trạng thái model, device, số danh mục
GET /	Thông tin API version và model

Hệ thống phân loại 32 danh mục thương mại điện tử phổ biến, được định nghĩa trong categories.py với mỗi danh mục gồm: ID số (0–31), tên nội bộ (ví dụ: electronics), tên hiển thị (ví dụ: "Electronics"), và mô tả chi tiết các sản phẩm thuộc danh mục đó. Các danh mục bao phủ đa dạng lĩnh vực từ Electronics, Fashion, Home, Food, đến Industrial & Commercial.

3.7 Thiết kế server trung gian

Server trung gian là thành phần kết nối then chốt trong kiến trúc hệ thống, chạy trên Node.js/Express tại port 3000. Thành phần này giải quyết sự khác biệt về giao thức giữa ESP32 (TCP) và trình duyệt (WebSocket/HTTP), đồng thời đóng vai trò điều phối request đến các dịch vụ AI.

Về phía thiết bị nhúng, server duy trì hai TCP server: TCP Camera Server (port 3001) nhận JPEG frame từ ESP32, parse header 4 byte chứa độ dài frame rồi broadcast qua WebSocket endpoint /camera đến trình duyệt; TCP Audio Server (port 3002) nhận stream 16-bit PCM, duy trì byte-alignment và broadcast qua WebSocket endpoint /audio.

Về phía trình duyệt, server cung cấp HTTP API Proxy chuyển tiếp request đến các dịch vụ AI: POST /api/process-voice gọi tuần tự Denoise Service rồi STT Service, POST /api/gen-description gọi Voice Rephraze Service, và POST /api/classify gọi Classification Service. Figure 3.7 minh họa kiến trúc server trung gian.

Thêm hình: Sơ đồ kiến trúc server trung gian — thể hiện ESP32 kết nối TCP (port 3001, 3002) đến Node.js, Node.js relay qua WebSocket (/camera, /audio) đến Browser, đồng thời proxy HTTP API đến 3 dịch vụ AI.

Hình 3.7: Kiến trúc server trung gian

Giao diện web tích hợp hiển thị video streaming real-time, biểu đồ amplitude âm thanh 30 bars, spectrogram trước và sau khử nhiễu thông qua thư viện WaveSurfer.js, cùng kết quả nhận dạng và phân loại sản phẩm.

Thêm hình: Ảnh chụp màn hình giao diện web — hiển thị các thành phần: video streaming từ camera, biểu đồ amplitude âm thanh, spectrogram trước/sau khử nhiễu, kết quả nhận dạng giọng nói, mô tả sản phẩm đã sinh, và danh sách top-K danh mục dự đoán.

Hình 3.8: Giao diện web hệ thống ShopNexus

CHƯƠNG 4

CÀI ĐẶT HỆ THỐNG

4.1 Cài đặt thiết bị nhúng ESP32

4.1.1 Môi trường phát triển

Firmware cho ESP32-CAM được phát triển trên Arduino IDE với ESP32 Board Package của Espressif. Các thư viện chính được sử dụng bao gồm: esp_camera.h cho điều khiển camera OV5640, driver/i2s.h cho giao tiếp I2S với microphone INMP441, và WiFi.h cho kết nối mạng. Firmware được flash qua UART với baudrate 115200.

4.1.2 Kết nối WiFi và khởi tạo

Khi khởi động, ESP32 thực hiện tuần tự các bước: kết nối WiFi đến access point được cấu hình sẵn, khởi tạo camera OV5640 với các tham số mặc định (QVGA, JPEG quality 12), cấu hình I2S cho microphone INMP441, và cuối cùng thiết lập kết nối TCP đến server trung gian. Nếu mất kết nối WiFi trong quá trình hoạt động, ESP32 tự động thử kết nối lại với khoảng cách 5 giây giữa các lần thử.

4.1.3 Cấu hình I2S

Bảng 4.1: Cấu hình I2S cho microphone INMP441

Tham số	Giá trị
Chế độ	I2S_MODE_MASTER | I2S_MODE_RX
Tần số lấy mẫu	16.000 Hz
Bit depth đầu vào	32-bit (I2S native)
Bit depth đầu ra	16-bit PCM (sau chuyển đổi)
Kênh	Mono (chỉ kênh trái)
Số DMA buffer	16 (tăng từ 8 để ổn định)
Kích thước buffer	1024 mẫu (64 ms mỗi buffer)
Throughput	32 KB/s ($\approx$ 256 kbps)

Số lượng DMA buffer được tăng từ 8 lên 16 sau quá trình thử nghiệm, nhằm giảm hiện tượng buffer overflow khi Core 1 bận xử lý camera capture. Với 16 buffer, hệ thống có thể chịu được độ trễ tối đa 16 $\times$ 64 ms = 1024 ms trước khi mất dữ liệu âm thanh.

4.1.4 Chuẩn hóa tín hiệu

Microphone INMP441 có độ phân giải 18-bit, đóng gói trong word 32-bit với dữ liệu nằm ở các bit cao. Quá trình chuẩn hóa bắt đầu bằng việc đọc mẫu 32-bit raw từ I2S DMA buffer, sau đó dịch phải 14 bit ($\mathrm{val} = \mathrm{raw_samples[i]} \gg 14$) để đưa 18 bit dữ liệu hữu ích về phạm vi 16-bit. Giá trị sau đó được cắt ngưỡng vào $[-32768, 32767]$ để tránh tràn khi có tín hiệu mạnh bất thường, và lưu dưới dạng $\mathrm{int16_t}$ để truyền qua TCP.

Việc dịch phải 14 bit (thay vì 16 bit) là lựa chọn có chủ đích: giữ lại toàn bộ 18 bit dữ liệu hữu ích thay vì mất 2 bit thấp, tận dụng tối đa độ phân giải của microphone. Trong thực tế, 2 bit thấp nhất thường chứa nhiễu lượng tử nên việc mất chúng qua bước cắt ngưỡng không ảnh hưởng đáng kể đến chất lượng âm thanh.

4.1.5 Pipeline streaming video

Camera OV5640 được cấu hình chụp JPEG với double buffering (2 frame buffer) để tối ưu throughput — trong khi một buffer đang được gửi qua TCP, camera có thể ghi vào buffer còn lại. XCLK mặc định 12 MHz, chất lượng JPEG có thể điều chỉnh từ 10 (chất lượng cao nhất, file lớn) đến 63 (chất lượng thấp nhất, file nhỏ).

Mỗi frame được truyền qua TCP port 3001 với giao thức tự định nghĩa: $[\mathrm{Length\ (4\ bytes\ LE)}][\mathrm{JPEG\ Data\ (variable)}]$. Header 4 byte chứa độ dài frame theo little-endian, cho phép server biết chính xác kích thước dữ liệu cần đọc. Buffer tĩnh 250 KB được cấp phát một lần trên PSRAM (hoặc heap nếu PSRAM không khả dụng) để tránh phân mảnh bộ nhớ do $\mathrm{malloc/free}$ lặp lại.

4.1.6 Giao thức điều khiển camera

Trình duyệt gửi lệnh điều khiển qua WebSocket đến server trung gian, server chuyển tiếp qua TCP đến ESP32 với format: $[\mathrm{0xA5}][\mathrm{ID\ (1\ byte)}][\mathrm{Value\ (1\ byte)}]$. Byte magic $\mathrm{0xA5}$ giúp ESP32 phân biệt lệnh điều khiển với dữ liệu thông thường trên cùng kết nối TCP.

ESP32 phản hồi cài đặt hiện tại qua packet với magic header $\mathrm{0xBEEF}$ theo format: $[\mathrm{0xBE}][\mathrm{0xEF}][\mathrm{ID}][\mathrm{Va}$ cho tất cả 21 tham số. Phản hồi này được gửi mỗi khi có thay đổi cài đặt, giúp giao diện web luôn đồng bộ với trạng thái thực tế của camera.

Thêm hình: Sơ đồ cấu trúc TCP frame — hiển thị header 4 bytes (Length, little-endian) theo sau bởi JPEG data có kích thước variable. Chú thích ví dụ: Length = 0x00 0x8C 0xA0 0x00 = 36.000 bytes.

Hình 4.1: Cấu trúc TCP frame cho video streaming

Bảng 4.2: Các lệnh điều khiển camera chính

ID	Tham số	Phạm vi
1	JPEG Quality	10–63 (thấp = chất lượng cao hơn)
2	Frame Size	QQVGA đến UXGA (14 mức)
3–5	Brightness, Contrast, Saturation	-2 đến $+2$
6–9	AWB, AEC, AGC, AEC2	0/1 (bật/tắt)
16–17	H-Mirror, V-Flip	0/1
20	XCLK Frequency	6/8/12/16/24 MHz
21	Frame Buffer Count	1–4

4.2 Cài đặt dịch vụ khử nhiễu

4.2.1 Huấn luyện trên VIVOS

Mô hình DTLN được huấn luyện trên bộ dữ liệu tiếng Việt VIVOS làm nguồn giọng nói sạch, kết hợp với dữ liệu nhiễu Microsoft DNS ở các mức SNR ngẫu nhiên để tạo cặp dữ liệu huấn luyện (audio nhiễu, audio sạch). Quá trình huấn luyện sử dụng đoạn audio 3 giây (phù hợp với độ dài trung bình 3–5 giây của VIVOS), batch size 32, và optimizer Adam với clipnorm 3.0 để tránh bùng nổ gradient. Learning rate ban đầu 10^{-3} được giảm tự động qua ReduceLROnPlateau khi validation loss không cải thiện, kết hợp Early Stopping sau 10 epoch không tiến triển để tránh overfitting. Table 4.3 tổng hợp cấu hình huấn luyện.

Model sau huấn luyện được lưu dưới dạng file HDF5 (DTLN_vivos_best.h5, 3,9 MB) chứa toàn bộ trọng số của khoảng 1,8 triệu tham số.

Bảng 4.3: Cấu hình huấn luyện DTLN trên VIVOS

Tham số	Giá trị	Giải thích
Dữ liệu sạch	VIVOS	Tiếng Việt, $>$ 1 GB
Dữ liệu nhiễu	Microsoft DNS	Kết hợp random SNR
Batch size	32	Cân bằng GPU/gradient
Độ dài đoạn	3 giây	Phù hợp VIVOS (3–5s)
Epoch tối đa	50	Kết hợp dừng sớm
Learning rate	10^{-3}	ReduceLROnPlateau
Early Stopping	10 epoch	Tránh overfitting
Optimizer	Adam, clipnorm=3.0	Tránh bùng nổ gradient

4.2.2 Cài đặt mô hình DTLN

Kiến trúc DTLN được triển khai trong file dtln_model.py bằng TensorFlow/Keras với các hyperparameter: 128 LSTM units, 2 lớp LSTM mỗi giai đoạn, dropout 0.25, encoder size 256, block length 512 mẫu và block shift 128 mẫu. Phương thức build_DTLN_model() xây dựng graph tính toán gồm: lớp STFT (framing + rFFT), Separation Kernel thứ nhất (2 LSTM + Dense + Sigmoid mask nhân với magnitude, kết hợp phase gốc, iFFT), lớp Conv1D encoder (chiếu sang 256 chiều), Separation Kernel thứ hai (2 LSTM + Dense + Sigmoid mask trong không gian đặc trưng), Conv1D decoder (256→512 với causal padding), và overlap-and-add để tái tạo tín hiệu thời gian.

Lớp InstantLayerNormalization tùy chỉnh thực hiện chuẩn hóa theo channel (axis=-1) với tham số gamma và beta có thể học được, áp dụng công thức $(x-\mu)/\sqrt{\sigma^2+\varepsilon}$ với $\varepsilon = 10^{-7}$.

4.2.3 Pipeline xử lý khử nhiễu

Khi nhận request tại POST /denoise, tầng API thực hiện validate: kiểm tra file tồn tại trong request, filename không rỗng, và extension là .wav (duy nhất được chấp nhận). Filename được sanitize qua werkzeug.utils.secure_filename trước khi lưu tạm với prefix input_ vào thư mục uploads/.

Tầng Service đọc file WAV qua thư viện SoundFile, tự động normalize sang float32 trong phạm vi $[-1,1]$. Nếu sample rate khác 16 kHz, hệ thống ghi log cảnh báo nhưng vẫn tiếp tục xử lý. File stereo được convert sang mono bằng cách lấy trung bình hai kênh (np.mean(audio, axis=1)). Tín hiệu sau đó được thêm batch dimension (np.expand_dims) và đưa qua model.predict(verbos để khử nhiễu. Đầu ra được np.clip vào $[-1,1]$ và ghi thành file WAV với prefix denoised_.

File kết quả được gửi về client qua send_file() với MIME type audio/wav. Cả file input và output tạm đều được xóa tự động sau khi response gửi xong thông qua callback response.call_on_close, đảm bảo cleanup ngay cả khi có lỗi xảy ra trong quá trình gửi.

4.2.4 Xử lý lỗi

Dịch vụ xử lý 5 loại lỗi HTTP: 400 (file thiếu, filename rỗng, extension không hợp lệ, audio format sai), 404 (file không tìm thấy), 413 (file vượt quá 50 MB — Flask tự xử lý qua MAX_CONTENT_LENGTH), và 500 (lỗi nội bộ). Tất cả response lỗi trả về JSON với trường error và message. File tạm luôn được cleanup kể cả trong trường hợp exception.

4.3 Cài đặt dịch vụ nhận dạng giọng nói

4.3.1 Speech-to-Text với PhoWhisper

Module STT sử dụng PhoWhisper-large (vinai/PhoWhisper-large), mô hình nhận dạng giọng nói tiếng Việt được phát triển bởi VinAI Research dựa trên kiến trúc OpenAI Whisper. Model được load thông qua HuggingFace Transformers pipeline (automatic-speech-recognition) khi server khởi động, yêu cầu khoảng 2 GB VRAM.

Khi nhận request tại endpoint POST /stt, dịch vụ validate định dạng file dựa trên extension (whitelist 9 định dạng: wav, mp3, flac, ogg, m4a, webm, mp4, mpeg, mpga), lưu file tạm thời với prefix temp_ bằng shutil.copyfileobj() để stream hiệu quả không tải toàn bộ file vào RAM, sau đó đưa qua pipeline PhoWhisper để nhận dạng. File tạm được xóa ngay sau khi xử lý xong, kể cả trong trường hợp xảy ra lỗi, để tránh tích tụ trên disk.

4.3.2 Sinh mô tả sản phẩm với GPT-4o-mini

Endpoint POST /gen nhận hai tham số: product_description (text từ STT hoặc người dùng nhập trực tiếp) và style (tên phong cách viết). Dịch vụ truy vấn SQLite để lấy mô tả chi tiết của phong cách, sau đó xây dựng prompt gồm hai phần.

System prompt thiết lập persona chuyên gia copywriter thương mại điện tử với các ràng buộc rõ ràng: phân tích văn bản voice-to-text lộn xộn, trích xuất thông tin cốt lõi sản phẩm (tên, chất liệu, màu sắc, tính năng, lợi ích, đối tượng), viết lại thành mô tả hoàn chỉnh, không bịa đặt thông tin, loại bỏ từ lấp ("ừm", "à") và lỗi phát âm, chỉ trả về mô tả sản phẩm thuần túy (không giải thích, không tiêu đề), độ dài lý tưởng 3–5 câu ngắn gọn nhưng thuyết phục.

User prompt cung cấp cấu trúc hai bước: bước 1 — xác định thông tin sản phẩm từ văn bản đầu vào, bước 2 — viết lại theo phong cách được chỉ định. Lời gọi API sử dụng AsyncOpenAI

client với model gpt-4o-mini, temperature 0.75 (cân bằng giữa sáng tạo và nhất quán) và max_tokens 600.

4.3.3 Quản lý phong cách viết

Phong cách viết được lưu trữ trong SQLite (styles.db) với schema gồm 3 trường: id (auto-increment), name (UNIQUE, NOT NULL) và description (NOT NULL). Khi khởi động, nếu bảng rỗng, hệ thống tự động khởi tạo 5 phong cách mặc định.

Người dùng có thể thêm phong cách mới qua POST /add-style với hai trường name và description. Ràng buộc UNIQUE trên name ngăn trùng lặp, trả về lỗi 400 nếu tên đã tồn tại. Tất cả truy vấn SQL sử dụng parameterized query để chống SQL injection. Endpoint GET /styles trả về danh sách tất cả phong cách hiện có để giao diện web hiển thị cho người dùng lựa chọn.

4.3.4 Công nghệ và dependencies

Dịch vụ yêu cầu Python $\geq$ 3.14 với các dependencies chính: FastAPI (web framework), OpenAI SDK (gọi GPT-4o-mini bất đồng bộ), HuggingFace Transformers + PyTorch (load và chạy PhoWhisper), python-dotenv (quản lý biến môi trường), python-multipart (xử lý file upload), và Uvicorn (ASGI server). Server lắng nghe tại port 5001 trên tất cả interfaces.

4.4 Cài đặt dịch vụ phân loại sản phẩm

4.4.1 Validation đầu vào

Dịch vụ sử dụng Pydantic v2 với ConfigDict để validate tất cả request. Đối với endpoint POST /classify, text đầu vào bị ràng buộc 1–2000 ký tự (Field(min_length=1, max_length=2000)) và tham số top_k giới hạn trong khoảng 1–32 (mặc định 5). Đối với endpoint batch, danh sách texts bị giới hạn 1–100 items. Request không hợp lệ bị từ chối trước khi đến logic xử lý, trả về lỗi 422 với chi tiết validation từ Pydantic. Mỗi schema đều kèm json_schema_extra chứa ví dụ minh họa, tự động hiển thị trên Swagger UI tại /docs.

4.4.2 Pipeline suy luận

Khi nhận request tại POST /classify, dịch vụ lấy pipeline classifier từ app.state và đo thời gian xử lý qua time.perf_counter(). Text đầu vào được đưa trực tiếp qua HuggingFace pipeline, nội bộ thực hiện: tokenize (tối đa 512 tokens qua AutoTokenizer), forward pass qua XLM-RoBERTa (12 lớp transformer, 768 hidden units), và softmax để sinh confidence score cho 32 danh mục. Kết quả raw được chuyển đổi qua hàm _to_prediction() — ánh xạ label text

sang category ID, internal name, display name qua categories.py, kèm confidence score. Nếu model trả về label không có trong danh sách 32 danh mục (trường hợp hiếm), hệ thống ghi log warning và trả về prediction với category_id=-1 và display name "Unknown".

Endpoint batch (POST /classify/batch) tận dụng khả năng batched inference thật sự của HuggingFace pipeline với batch_size=32 — tất cả texts trong request được xử lý trong một forward pass duy nhất qua transformer, hiệu quả hơn đáng kể so với gọi lần lượt từng text. Thời gian suy luận cho từng item trong batch được đặt là 0.0 (chỉ đo tổng thời gian toàn batch) vì tokenization và forward pass được thực hiện chung.

4.4.3 Containerization

Dịch vụ được đóng gói trong Docker container dựa trên python:3.11-slim với single worker Uvicorn (1 worker thay vì N workers để tránh nhân bản model 1,1 GB trong RAM cho mỗi worker). Docker Compose cấu hình volume model-cache ánh xạ đến HF_HOME để cache model đã download từ HuggingFace Hub, tránh download lại khoảng 1,1 GB mỗi lần restart container. Health check chạy mỗi 30 giây với start period 90 giây (thời gian chờ download model lần đầu), đánh dấu container unhealthy nếu 3 lần check liên tiếp thất bại.

4.4.4 Hiệu năng

Table 4.4 trình bày benchmark hiệu năng trên CPU và GPU. Single inference trên CPU đạt khoảng 45 ms (22 req/s), trong khi trên GPU giảm xuống 15 ms (66 req/s). Batch processing cho throughput cao hơn đáng kể: 55 items/s (CPU) và 125 items/s (GPU) nhờ amortize overhead của tokenization và model loading trên nhiều items.

Bảng 4.4: Benchmark hiệu năng phân loại sản phẩm

Cấu hình	Latency	Throughput
Single inference (CPU)	≈ 45 ms	22 req/s
Batch 10 items (CPU)	≈ 180 ms	55 items/s
Single inference (GPU)	≈ 15 ms	66 req/s
Batch 100 items (GPU)	≈ 800 ms	125 items/s

CHƯƠNG 5

KẾT QUẢ VÀ ĐÁNH GIÁ

5.1 Kết quả đạt được

Sau quá trình phát triển và tích hợp, hệ thống ShopNexus đã hoàn thành đầy đủ các mục tiêu đề ra. Table 5.1 tổng hợp kết quả chính của từng thành phần.

Bảng 5.1: Tổng hợp kết quả các thành phần

Thành phần	Kết quả
ESP32-CAM	Dual-core: audio 16 kHz liên tục + video JPEG streaming, điều khiển camera real-time (21 tham số)
Khử nhiễu DTLN	1,8M tham số, 3,9 MB, 32 ms latency, huấn luyện trên VIVOS tiếng Việt
Voice Rephraze	STT qua PhoWhisper-large (local) + sinh mô tả qua GPT-4o-mini, 5+ phong cách viết
Phân loại sản phẩm	Accuracy 90,14%, F1 90,00%, 32 danh mục, 45 ms/request (CPU)
Server trung gian	Relay TCP↔WebSocket, proxy API, spectrogram visualization

5.2 Kết quả thiết bị nhúng ESP32

Thiết bị ESP32-CAM hoạt động ổn định trong môi trường thử nghiệm với kết nối WiFi trong phạm vi 10–15 mét. Kiến trúc dual-core đảm bảo stream audio 16 kHz không bị gián đoạn ngay cả khi camera đang chụp ở độ phân giải cao (UXGA 1600×1200). Thời gian khởi động từ lúc cấp nguồn đến khi sẵn sàng stream là khoảng 3–5 giây, bao gồm kết nối WiFi và khởi tạo camera.

Giao thức điều khiển camera cho phép thay đổi 21 tham số real-time từ giao diện web, với thời gian phản hồi dưới 100 ms. Double buffering cho video streaming đạt frame rate ổn định 10–15 FPS ở QVGA và 5–8 FPS ở VGA.

5.3 Kết quả khử nhiễu DTLN

Mô hình DTLN sau khi huấn luyện trên dữ liệu VIVOS kết hợp nhiễu Microsoft DNS đạt khả năng khử nhiễu hiệu quả trên giọng nói tiếng Việt. Với chỉ 1,8 triệu tham số và kích thước 3,9 MB, mô hình xử lý mỗi khung audio (32 ms) trong thời gian tương đương, đảm bảo xử lý real-time.

Thêm hình: So sánh spectrogram trước và sau khử nhiễu — hiển thị 2 spectrogram song song: (a) audio gốc có nhiễu nền (quạt, tiếng ồn đường phố), (b) audio sau khi qua DTLN với nhiễu nền được loại bỏ rõ rệt, giọng nói được giữ nguyên.

Hình 5.1: So sánh spectrogram trước và sau khử nhiễu DTLN

5.4 Kết quả nhận dạng giọng nói và sinh mô tả

Dịch vụ Voice Rephraze sử dụng PhoWhisper-large (VinAI) chạy local cho kết quả nhận dạng tiếng Việt chính xác, kể cả với giọng vùng miền (Bắc, Trung, Nam). Việc chạy local loại bỏ phụ thuộc cloud API cho STT, giảm chi phí vận hành và latency mạng. Module sinh mô tả sản phẩm qua GPT-4o-mini tạo ra văn bản mạch lạc, tự nhiên, phù hợp với phong cách viết được chọn.

5.5 Kết quả phân loại sản phẩm

Mô hình XLM-RoBERTa đạt accuracy 90,14% và F1-score 90,00% trên tập kiểm thử 32 danh mục. Thời gian suy luận trung bình 45 ms trên CPU cho single inference, đủ nhanh cho ứng dụng real-time. Table 5.2 trình bày chi tiết hiệu năng trên các cấu hình khác nhau.

Thêm hình: Ví dụ minh họa kết quả — hiển thị: (1) text STT gốc (lộn xộn, có từ lấp), (2) mô tả sản phẩm đã sinh theo phong cách "Chuyên nghiệp", (3) mô tả theo phong cách "Thân thiện". Cho thấy sự khác biệt giữa các phong cách viết.

Hình 5.2: Ví dụ kết quả nhận dạng giọng nói và sinh mô tả sản phẩm

Bảng 5.2: Hiệu năng phân loại trên các cấu hình

Cấu hình	Latency	Throughput
Single inference (CPU)	$\approx$ 45 ms	22 req/s
Batch 10 items (CPU)	$\approx$ 180 ms	55 items/s
Single inference (GPU)	$\approx$ 15 ms	66 req/s
Batch 100 items (GPU)	$\approx$ 800 ms	125 items/s

Thêm hình: Biểu đồ top-5 predictions cho một ví dụ cụ thể — bar chart hiển thị tên danh mục và confidence score (ví dụ: "Electronics" 87%, "Computers" 6%, "Mobile Phones" 3%, ...).

Hình 5.3: Ví dụ kết quả phân loại sản phẩm với top-5 predictions

5.6 Kết quả tích hợp hệ thống

Toàn bộ pipeline hoạt động liền mạch từ khi người dùng bắt đầu nói đến khi kết quả hiển thị trên giao diện web. Thời gian xử lý end-to-end trung bình khoảng 5–8 giây cho một đoạn mô tả 10–15 giây, bao gồm: truyền audio từ ESP32 (real-time), khử nhiễu DTLN ($\approx$ 1 giây cho đoạn 10 giây), STT ($\approx$ 2–3 giây), sinh mô tả ($\approx$ 1–2 giây), và phân loại ($\approx$ 45 ms).

Thêm hình: Ảnh chụp toàn bộ giao diện web đang hoạt động — hiển thị đồng thời: video stream từ camera, waveform âm thanh, spectrogram trước/sau khử nhiễu, kết quả STT, mô tả sản phẩm đã sinh, và danh sách top-K danh mục. Chú thích các vùng chức năng.

Hình 5.4: Giao diện web hệ thống ShopNexus trong quá trình hoạt động

5.7 Ưu điểm

- **Pipeline end-to-end hoàn chỉnh:** Từ thu âm ESP32 $\rightarrow$ khử nhiễu $\rightarrow$ nhận dạng giọng nói $\rightarrow$ sinh mô tả $\rightarrow$ phân loại sản phẩm, hoạt động liền mạch không cần can thiệp thủ công.
- **DTLN nhỏ gọn và hiệu quả:** Chỉ 3,9 MB (1,8M tham số), xử lý real-time 32 ms, phù hợp triển khai trên thiết bị tài nguyên hạn chế. Huấn luyện trực tiếp trên dữ liệu tiếng Việt VIVOS đảm bảo hiệu quả với ngôn ngữ mục tiêu.
- **Dual-domain denoising:** Kết hợp STFT (miền tần số) và Conv1D Encoder (miền đặc trưng học được), khử nhiễu toàn diện hơn so với phương pháp đơn miền. Giai đoạn 1 loại bỏ nhiễu tổng thể, giai đoạn 2 tinh chỉnh và phục hồi chi tiết.
- **Phân loại chính xác:** XLM-RoBERTa đạt 90,14% accuracy trên 32 danh mục, hỗ trợ đa ngôn ngữ nhờ nền tảng cross-lingual — xử lý tốt cả mô tả tiếng Việt lẫn tên sản phẩm tiếng Anh.

- **Dual-core ESP32 tối ưu:** Tách riêng audio/video trên 2 core đảm bảo không gián đoạn stream. 16 DMA buffer cho audio tạo vùng đệm 1024 ms, chống buffer underrun hiệu quả.
- **Sinh mô tả đa phong cách:** Hỗ trợ 4+ phong cách viết (văn minh, chuyên nghiệp, thân thiện, văn hóa dân tộc), mở rộng được qua API. Prompt engineering xử lý tốt text lộn xộn, từ lấp từ STT.
- **Kiến trúc microservices:** Mỗi dịch vụ phát triển, triển khai và scale độc lập. Có thể thay thế từng thành phần (ví dụ: đổi STT engine) mà không ảnh hưởng các thành phần khác.

5.8 Hạn chế

- **DTLN chỉ mono channel:** Chưa khai thác beamforming hay mảng microphone để tăng SNR đầu vào, hạn chế hiệu quả trong môi trường nhiều nguồn nhiễu.
- **Phụ thuộc cloud cho sinh mô tả:** Module sinh mô tả sản phẩm sử dụng OpenAI GPT-4o-mini API, yêu cầu kết nối internet và phát sinh chi phí vận hành. Module STT (PhoWhisper) đã chạy local nhưng toàn bộ pipeline vẫn cần internet cho bước sinh mô tả.
- **DTLN nhạy cảm nhiễu phi tĩnh:** Hiệu năng giảm đáng kể với nhiễu biến đổi nhanh (tiếng nói chồng chéo, âm nhạc phức tạp) do LSTM cần thời gian thích nghi với pattern nhiễu mới.
- **Pha không tối ưu:** Giai đoạn 1 của DTLN tái sử dụng pha gốc thay vì ước lượng pha sạch, có thể gây artifact âm thanh khi nhiễu nặng làm biến dạng pha ban đầu.
- **ESP32 phụ thuộc WiFi:** Thiết bị không có cơ chế buffer offline — nếu mất WiFi, dữ liệu audio/video bị mất hoàn toàn thay vì được lưu trữ tạm để gửi lại sau.
- **Chưa có hệ thống đánh giá tự động:** Kết quả STT và sinh mô tả chưa được đánh giá bằng metrics chuẩn (WER, BLEU) do thiếu bộ test benchmark cho tiếng Việt trong ngữ cảnh thương mại điện tử.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã xây dựng thành công hệ thống nhúng hỗ trợ bán hàng thông minh ShopNexus, bao gồm một pipeline xử lý AI hoàn chỉnh từ thu thập dữ liệu đến hiển thị kết quả. Các kết quả chính đạt được:

Về thiết bị nhúng, ESP32-CAM khai thác hiệu quả kiến trúc dual-core để thu đồng thời âm thanh 16 kHz và video JPEG mà không xảy ra hiện tượng buffer underrun. Giao thức điều khiển camera tự thiết kế cho phép thay đổi 21 tham số real-time từ giao diện web.

Về khử nhiễu, mô hình DTLN được huấn luyện trên dữ liệu tiếng Việt VIVOS cho kết quả khử nhiễu tốt với chỉ 1,8 triệu tham số (3,9 MB) và latency 32 ms, phù hợp yêu cầu real-time. Kiến trúc hai giai đoạn (STFT + Conv1D Encoder) khử nhiễu toàn diện hơn phương pháp đơn miền.

Về nhận dạng và sinh mô tả, dịch vụ Voice Rephraze kết hợp PhoWhisper-large (STT local) và GPT-4o-mini (sinh mô tả) tạo ra mô tả sản phẩm chất lượng với 4+ phong cách viết. Prompt engineering xử lý hiệu quả text lộn xộn từ STT, bỏ qua từ lấp và lỗi phát âm.

Về phân loại sản phẩm, XLM-RoBERTa đạt accuracy 90,14% và F1-score 90,00% trên 32 danh mục thương mại điện tử, với thời gian suy luận 45 ms trên CPU — đáp ứng yêu cầu ứng dụng thực tế.

Nhìn tổng thể, hệ thống đã chứng minh tính khả thi của việc kết hợp thiết bị nhúng giá rẻ (ESP32, giá khoảng 5 USD) với các dịch vụ AI hiện đại để tạo ra giải pháp hỗ trợ bán hàng tự động. Người bán chỉ cần mô tả sản phẩm bằng giọng nói, hệ thống tự động xử lý toàn bộ quy trình còn lại — tiết kiệm thời gian và nâng cao chất lượng mô tả sản phẩm.

6.2 Hướng phát triển

Dựa trên kết quả và hạn chế đã phân tích, nhóm đề xuất các hướng phát triển sau cho hệ thống ShopNexus:

- **Thay thế GPT-4o-mini bằng LLM local:** Triển khai mô hình ngôn ngữ local (ví dụ: Vistral, Qwen2) để sinh mô tả sản phẩm, loại bỏ hoàn toàn phụ thuộc cloud API và chi phí

vận hành OpenAI. Kết hợp với PhoWhisper-large đã chạy local, toàn bộ pipeline sẽ hoạt động offline.

- **Bổ sung multi-microphone beamforming:** Sử dụng 2–4 microphone INMP441 kết hợp thuật toán beamforming (delay-and-sum hoặc MVDR) để tăng SNR đầu vào trước khi đưa vào DTLN, cải thiện đáng kể chất lượng trong môi trường nhiều nhiễu.
- **Cải tiến DTLN:** Thêm windowing function (Hann hoặc Hamming) trong bước STFT để giảm spectral leakage. Nghiên cứu ước lượng pha sạch thay vì tái sử dụng pha gốc ở giai đoạn 1, giảm artifact âm thanh.
- **Tối ưu phân loại cho tiếng Việt:** Fine-tune thêm mô hình XLM-RoBERTa trên dữ liệu mô tả sản phẩm tiếng Việt thực tế từ các sàn thương mại điện tử Việt Nam (Shopee, Lazada), nâng cao accuracy cho các danh mục đặc thù thị trường nội địa.
- **Triển khai edge inference:** Chuyển DTLN sang TensorFlow Lite và triển khai trực tiếp trên Raspberry Pi 4 hoặc thiết bị edge tương đương, giảm latency mạng và cho phép hoạt động semi-offline.
- **Buffer offline cho ESP32:** Bổ sung cơ chế lưu trữ tạm audio vào thẻ microSD khi mất kết nối WiFi, tự động gửi lại khi kết nối được khôi phục — đảm bảo không mất dữ liệu trong môi trường mạng không ổn định.
- **Hỗ trợ đa nền tảng:** Phát triển ứng dụng mobile (Android/iOS) song song với giao diện web, cho phép người bán sử dụng microphone trên điện thoại khi không có thiết bị ESP32.

TÀI LIỆU THAM KHẢO

[1] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.

[2] Espressif Systems. *ESP-IDF Programming Guide*, 2024. Version 5.3.

[3] Espressif Systems. *ESP32 Technical Reference Manual*, 2024. Version 5.2.

[4] Ian Fette and Alexey Melnikov. The WebSocket protocol. RFC 6455, 2011.

[5] Google Brain Team. TensorFlow: An end-to-end open source machine learning platform, 2024.

[6] InvenSense (TDK). *INMP441 Omnidirectional Microphone with Bottom Port and I2S Digital Output*, 2014. Datasheet Rev 1.0.

[7] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[8] Hieu-Thi Luong and Hai-Quan Vu. A non-expert Kaldi recipe for Vietnamese speech recognition system. In *Proceedings of the Third International Workshop on Worldwide Language Service Infrastructure and Second Workshop on Open Infrastructures and Analysis Frameworks for Human Language Technologies*, pages 51–55, 2016.

[9] Thanh-Thien Nguyen, Quang-Phuoc Nguyen, Thi-Nhung Nguyen, Cong-Duy Nguyen, and Dat Quoc Nguyen. PhoWhisper: Automatic speech recognition for Vietnamese. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.

[10] OmniVision Technologies. *OV5640 Camera Module Software Application Notes*, 2012. Version 2.0.

[11] OpenAI. GPT-4o mini: Advancing cost-efficient intelligence, 2024.

[12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library, 2024.

[13] Philips Semiconductors. I2S bus specification, 1986. Revised 1996.

[14] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, pages 28492–28518, 2023.

[15] Sebastián Ramírez. FastAPI: Modern, fast web framework for building APIs with Python, 2024.

[16] Chandan K.A. Reddy, Vishak Gopal, Ross Cutler, Ebrahim Beyrami, Roger Cheng, Harishchandra Dubey, Sergiy Matusevych, Robert Aichner, Ashkan Aazami, Sebastian Braun, et al. The INTERSPEECH 2020 deep noise suppression challenge: Datasets, subjective testing framework, and challenge results. In *Proceedings of Interspeech 2020*, pages 2492–2496, 2020.

[17] Armin Ronacher and Pallets Projects. Flask: A lightweight WSGI web application framework, 2024.

[18] Nils L. Westhausen and Bernd T. Meyer. Dual-signal transformation LSTM network for real-time noise suppression. In *Proceedings of Interspeech 2020*, pages 2477–2481, 2020.

[19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. HuggingFace Transformers: State-of-the-art machine learning for PyTorch, TensorFlow, and JAX, 2024.